МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ

«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

ФАКУЛЬТЕТ ІНФОРМАТИКИ ТА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ
КАФЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

ЛАБОРАТОРНА РОБОТА №3

з дисципліни «Бази даних»

на тему:

«Маніпулювання даними SQL (OLTP-транзакції та DML)»

Предметна область: CRM-система турнірів з більярдного спорту (Billiards Tournament CRM)

Виконав:
студент групи ІМ-55
Присяжний С.О.

Київ — 2026

1. Мета та завдання лабораторної роботи

Мета роботи: практичне засвоєння методів маніпулювання даними за допомогою мови SQL DML у режимі Online Transaction Processing (OLTP) у середовищі СУБД PostgreSQL 18.x; проектування, оптимізація та верифікація транзакцій бізнес-логіки турнірної більярдної системи (реєстрація користувачів, ведення карток спортсменів, проведення фінансових платежів, зміна станів столів, фіксація результатів матчів та безпечне видалення).

Основні завдання:

1. **SELECT-запити:** побудувати вибірки з точковою та інтервальною фільтрацією (`WHERE`, `INTERVAL`), багатотабличними з'єднаннями (`INNER JOIN`), сортуванням (`ORDER BY`) та скалярною агрегацією (`COUNT`).
2. **INSERT-запити:** реалізувати вставку даних у нормалізовані таблиці з використанням підзапитів для визначення зовнішніх ключів та клаузи `RETURNING` для отримання згенерованих ідентифікаторів.
3. **UPDATE-запити:** виконати цільові оновлення стану сутностей (підтвердження оплати реєстрацій, зайнятість обладнання, перерахунок рейтингових балів) із суворим дотриманням селективності умов.
4. **DELETE-запити:** продемонструвати безпечне видалення записів (скасування несплачених заявок, видалення тимчасових облікових записів) з контролем посилальної цілісності.
5. **Верифікація в СУБД:** зафіксувати результати виконання запитів у графічному клієнті pgAdmin 4.

2. Опис бізнес-сценаріїв OLTP у системі турнірного більярду

OLTP (Online Transaction Processing) фокусується на швидкій обробці великої кількості атомарних транзакцій. Для CRM-системи більярдного спорту ключовими є такі транзакційні сценарії:

- **Швидка диспетчеризація та пошук:** пошук гравців певної кваліфікації (наприклад, ЗМС) для жеребкування, отримання переможця конкретного матчу для оновлення сітки в реальному часі.
- **Онбординг та реєстрація:** створення профілю спортсмена, спортивної картки та подання заявки на турнір зі статусом `PENDING`.
- **Турнірний процесинг:** підтвердження банківської оплати (переведення статусу в `PAID`), фіксація зайнятості столу (`OCCUPIED`), динамічне нарахування рейтингу переможцю (+30 балів).
- **Скасування операцій:** видалення неоплачених заявок та очищення тестових записів.

3. Детальний опис та виконання SQL-запитів DML

3.1. Запити вибірки даних (SELECT)

Запит 1.1: Відбір користувачів віком понад 30 років

Формування списку спортсменів сеньйорської категорії за допомогою інтервальної дати `now() - INTERVAL '30 years'`:

```
select
```

```
        user_id,
        concat(first_name, ' ', last_name) as full_name,
        email,
        date_of_birth
from users
where date_of_birth < now() - INTERVAL '30 years';
```

Запит 1.2: Гравці зі званням ЗМС, відсортовані за рейтингом

Отримання списку спортсменів вищої категорії (`sports_title = 'HONORED_MASTER_OF_SPORTS'`) із сортуванням за спаданням кваліфікації:

```
select
        p.player_id,
        u.first_name,
        u.last_name,
        p.rating,
        p.sports_title
from player p
join users u on p.person_id = u.user_id
where sports_title = 'HONORED_MASTER_OF_SPORTS'
        and rating is not null
order by rating desc;
```

Запит 1.3: Визначення переможця фінального поєдинку (match_id = 3)

З'єднання таблиць `match_participant`, `player` та `users` для отримання даних тріумфатора фіналу:

```
select
        mp.match_participant_id,
        u.first_name,
        u.last_name,
        mp.match_id,
        mp.player_id,
        mp.team_id,
        mp.frames_won,
        mp.points,
        mp.side,
        mp.is_winner
from match_participant mp
join player p on mp.player_id = p.player_id
join users u on p.person_id = u.user_id
where match_id = 3
        and is_winner = true;
```

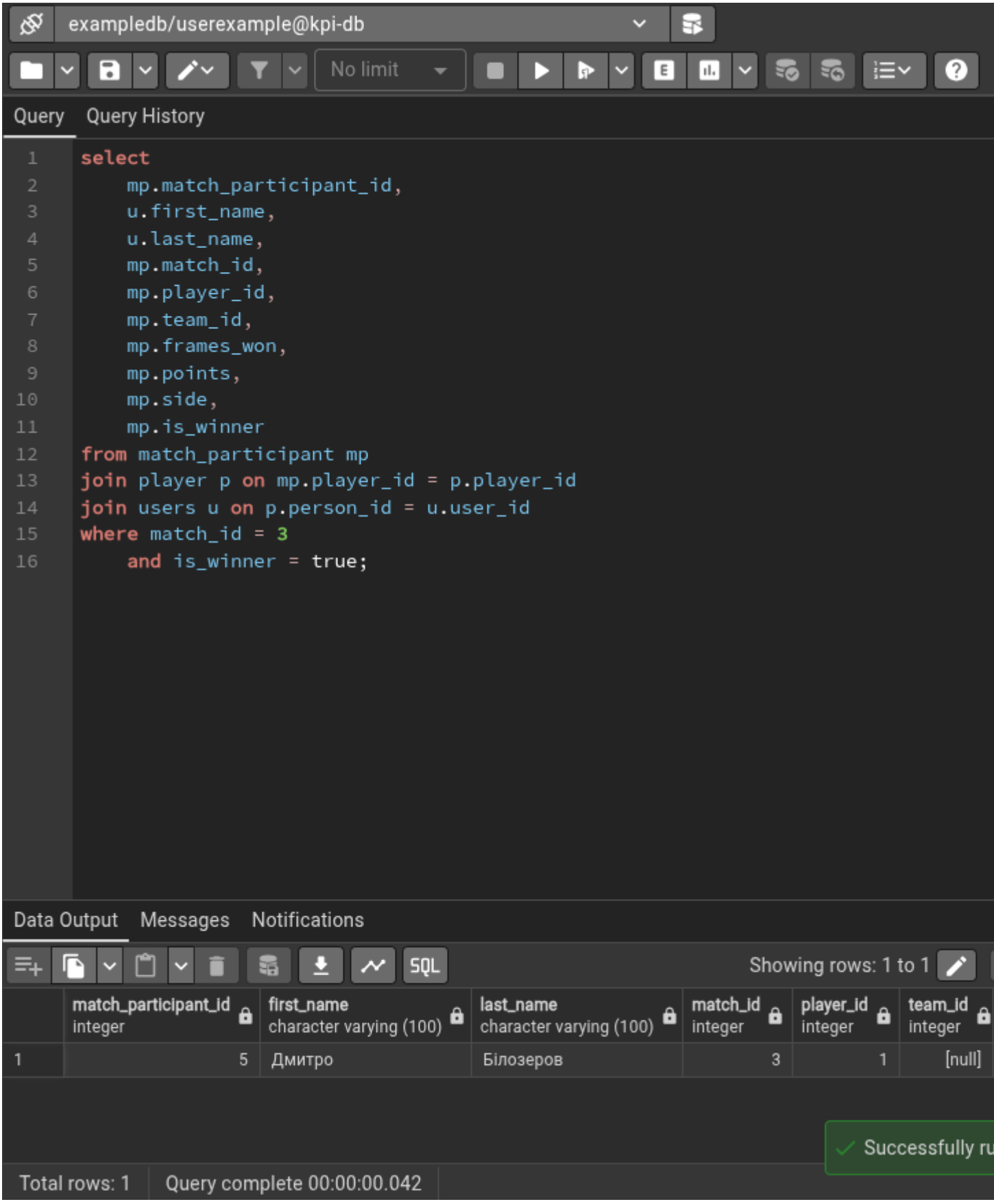


Рис. 1. Результат виконання SELECT-запиту визначення переможця матчу в pgAdmin 4

Результат: Запит повернув 1 рядок: `match_participant_id = 5`, переможець — **Дмитро Білозеров** (`player_id = 1`), який здобув 7 переможних партій (`frames_won = 7`) на стороні `SIDE_1`. Час виконання: 42 мс.

Запит 1.4: Підрахунок підтверджених реєстрацій на турнір №1

```
select count(*)
from tournament_registration
where tournament_id = 1
        and payment_status = 'PAID';
```

3.2. Запити додавання даних (INSERT INTO)

Запит 2.1: Онбординг нового користувача системи

Створення облікового запису спортсменки Діани Міронової з отриманням згенерованого `user_id` через `RETURNING`:

```
insert into users (first_name, last_name, email, password_hash, date_of_birth)
values (
        'Діана',
        'Міронова',
        'd.mironova@example.com',
        '$2b$12$LJ3m4ys3Lk0TDcUOqMRqGeAEIM4fP0Fv3sSzfMxHU0v4munYE2mKe',
        '1996-05-09'
)
returning user_id, first_name, last_name, email, date_of_birth;
```

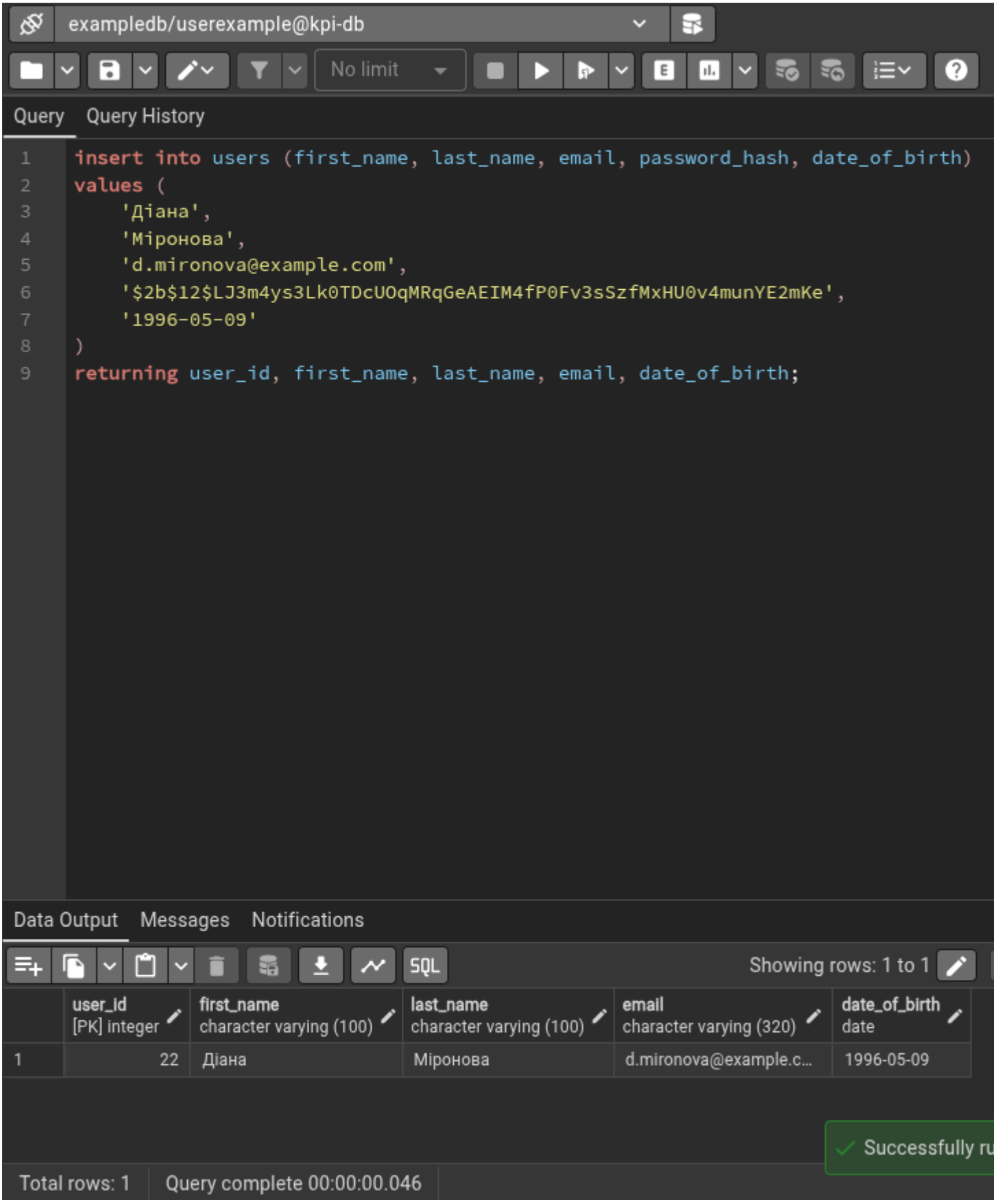


Рис. 2. Результат виконання INSERT-запиту з клаузою RETURNING в pgAdmin 4

Результат: Успішно створено запис користувача з ідентифікатором `user_id = 22`. Час виконання: 46 мс.

Запит 2.2: Оформлення спортивної картки гравця ЗМС

Динамічний пошук `person_id` через підзапит за унікальною адресою `email`:

```
insert into player (person_id, rating, sports_title)
values (
        (select user_id from users where email = 'd.mironova@example.com'),
        2450,
        'HONORED_MASTER_OF_SPORTS'
)
returning player_id, person_id, rating, sports_title;
```

Запит 2.3: Подання заявки на турнір №2

```
insert into tournament_registration (tournament_id, player_id, team_id, payment_status, prize_won)
values (
        2,
        (select p.player_id from player p join users u on p.person_id = u.user_id where u.email =
'd.mironova@example.com'),
        null,
        'PENDING',
        0.00
)
returning tournament_registration_id, tournament_id, player_id, payment_status;
```

3.3. Запити оновлення даних (UPDATE)

Запит 3.1: Підтвердження оплати реєстраційного внеску (PENDING → PAID)

Переведення статусу оплати в підтверджений стан із контролем через реляційний підзапит:

```
update tournament_registration
set payment_status = 'PAID'
where tournament_id = 2
        and player_id = (select p.player_id from player p join users u on p.person_id = u.user_id
                    where u.email = 'mironova1@example.com')
returning tournament_registration_id, tournament_id, player_id, payment_status;
```

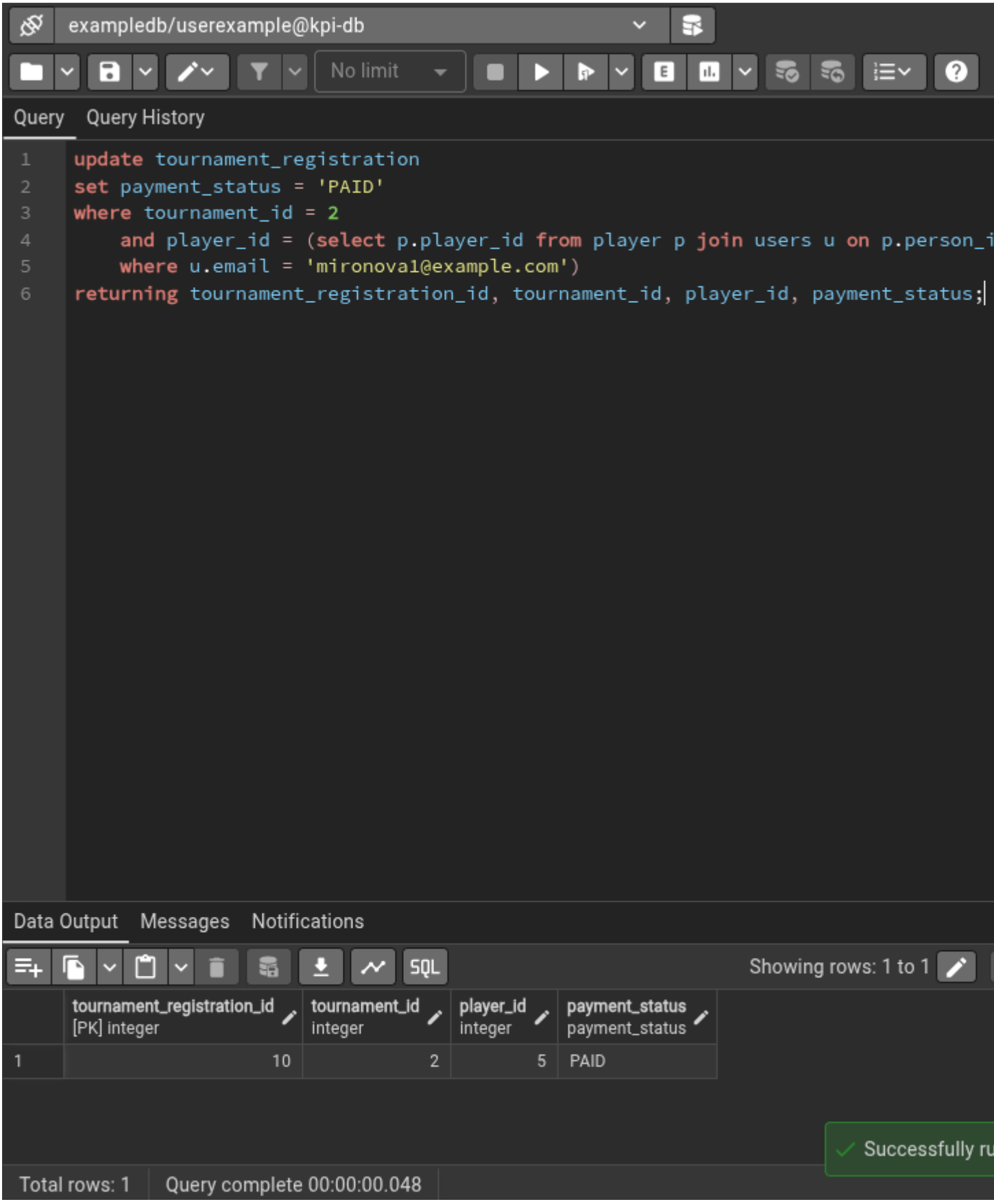


Рис. 3. Результат виконання UPDATE-запиту зміни статусу оплати в pgAdmin 4

Результат: Запис `tournament_registration_id = 10` для `player_id = 5` успішно переведено в статус `PAID`. Час виконання: 48 мс.

Запит 3.2: Диспетчеризація столу (призначення на матч)

```
update billiard_table
set table_status = 'OCCUPIED'
where table_id = 1
returning table_id, table_number, table_status;
```

Запит 3.3: Нарахування рейтингових очок переможцю фіналу (+30 очок)

```
update player
set rating = rating + 30
where player_id = 1
returning player_id, rating;
```

3.4. Запити видалення даних (DELETE)

Запит 4.1: Скасування неоплаченої реєстрації гравця

Безпечне видалення рядка з конкретною комбінацією умов (`tournament_id = 2 AND player_id = 2 AND payment_status = 'PENDING'`):

```
delete from tournament_registration
where tournament_id = 2
        and player_id = 2
        and payment_status = 'PENDING'
returning tournament_registration_id, tournament_id, player_id, payment_status;
```

Запит 4.2: Створення та видалення тестового користувача

Комплексна перевірка життєвого циклу запису: вставка тестового користувача та його миттєве видалення за унікальним email:

```
insert into users (first_name, last_name, email, password_hash, date_of_birth)
values (
        'Тест',
        'Тестовий',
        'test_delete@example.com',
        '$2b$12$LJ3m4ys3Lk0TDcUOqMRqGeAEIM4fP0Fv3sSzfMxHU0v4munYE2mKe',
        '2000-01-01'
)
returning user_id, first_name, last_name, email;

--- видалення користувача за email
delete from users
where email = 'test_delete@example.com'
returning user_id, first_name, last_name, email;
```

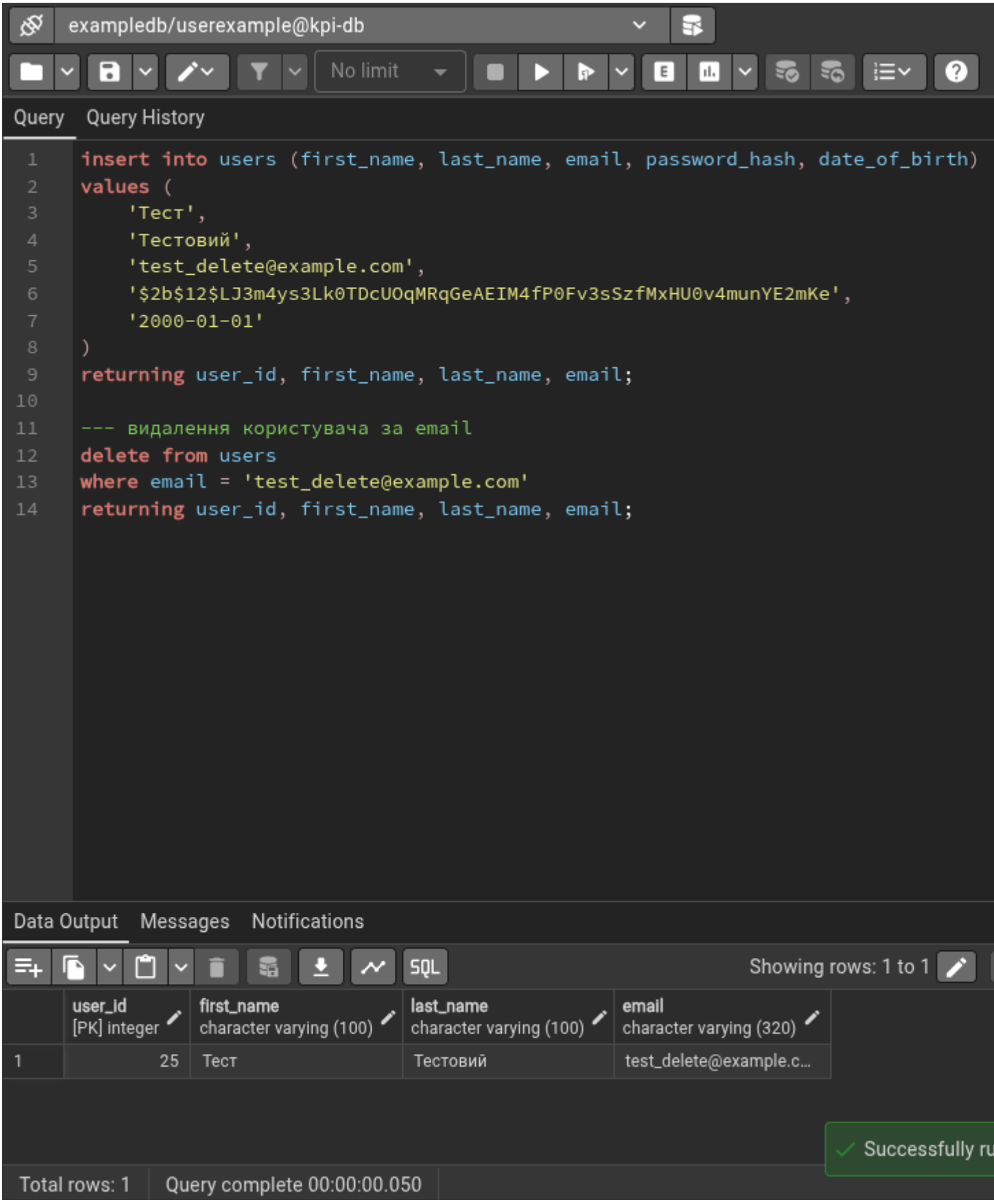


Рис. 4. Результат виконання DELETE-запиту з клаузою RETURNING в pgAdmin 4

Результат: Тестовий користувач був згенерований з `user_id = 25`, а наступною командою видалений. Клауза `RETURNING` повернула параметри видаленого рядка, підтверджуючи факт видалення саме цільового

запису. Час виконання: 50 мс.

4. Аналіз забезпечення цілісності та транзакційної безпеки

> **Ключові висновки щодо транзакційної надійності:**

- **Селективність WHERE:** жоден оператор `UPDATE` або `DELETE` не виконується без точного предикату, що запобігає масовій ненавмисній модифікації таблиць.
- **Переваги RETURNING:** дозволяє клієнту миттєво отримати актуальний стан кортежу в рамках єдиного мережевого запиту (RTT), усуваючи ризик стану гонитви (race condition).
- **Підзапити замість статичних ключів:** використання `(SELECT user_id FROM users WHERE email = ...)` забезпечує відмовостійкість скриптів до змін згенерованих ідентифікаторів.
- **Посилальна цілісність:** наявність обмежень `ON DELETE RESTRICT` на ключових зовнішніх ключах захищає історію турнірів від пошкодження у разі спроби видалення активних гравців.

5. Висновки

У ході виконання Лабораторної роботи №3 було успішно відпрацьовано технологію маніпулювання даними SQL DML у режимі OLTP на базі PostgreSQL 18.x. Створені запити повністю покривають ключові операційні задачі предметної області, продемонстровано ефективне використання підзапитів, інтервалів дат та конструкції `RETURNING`. Результати підтверджені даними графічного інтерфейсу pgAdmin 4.